The Journal of Open Source Software

NebulaTextures: A Stellarium Plugin for Custom Deep-Sky Textures and Online Astrometric Registration

WANG Siliang [1,2]

1 Zhanjiang Administration College, Guangdong, China **2** Huazhong University of Science and Technology, Hubei, China

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 11 June 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

NebulaTextures is a powerful and extensible open-source plugin for the Stellarium planetarium software. It dramatically simplifies and enriches the experience of visualizing and integrating deep-sky object (DSO) imagery provided by users. These images may include astrophotographs taken by amateur astronomers, hand-drawn or digitally illustrated astronomical sketches, or artistic renderings of celestial phenomena. The plugin enables users to insert their personalized visual content into Stellarium's virtual sky in a precise and astrometrically accurate manner.

Statement of Need

While Stellarium provides an expansive and curated catalog of built-in nebula and DSO textures, it lacks native support for user-defined, dynamically inserted visual assets. The manual process of aligning astrophotographs or drawings to celestial coordinates is technically demanding and error-prone, with limited user interface support. As a result, even experienced users often refrain from customizing their Stellarium installations.

This work presents NebulaTextures, a plugin that addresses this gap by offering an intuitive workflow for image import, automatic astrometric registration, and seamless integration into Stellarium's rendering engine. It empowers users to easily enhance their Stellarium experience with their own visual contributions.

Astronomical Context

Accurately placing astronomical images in a virtual sky relies on celestial coordinates, Right Ascension (RA) and Declination (Dec). Many images include WCS (World Coordinate System) data in their FITS headers, mapping pixel positions to sky coordinates. To obtain this mapping when absent, **astrometric registration** (or plate-solving) analyzes star patterns in the image and matches them to catalogs. The plugin automates this process via the Astrometry.net service, enabling precise sky alignment for user-submitted images.

System Design

NebulaTextures integrates with Stellarium using its established plugin infrastructure. The plugin's design is modular, with key components handling specific tasks:

Component	Purpose
`NebulaTextures`	Core plugin controller, manages hooks, rendering calls, and state
`NebulaTexturesDialog`	Provides the graphical user interface for user interaction, preview, and configuration
`PlateSolver`	Handles communication with the Astrometry.net API, submitting images and processing results
`SkyCoords`	Manages WCS transformations and the logic for projecting pixel coordinates to celestial positions
`TextureConfigManager`	Responsible for parsing and managing the JSON-based configuration files that define texture groups and properties
`TileManager`	Loads, blends, and displays textures using Stellarium's internal tile system for efficient rendering

The typical workflow for a user integrating an image is as follows chart:

DRAFT

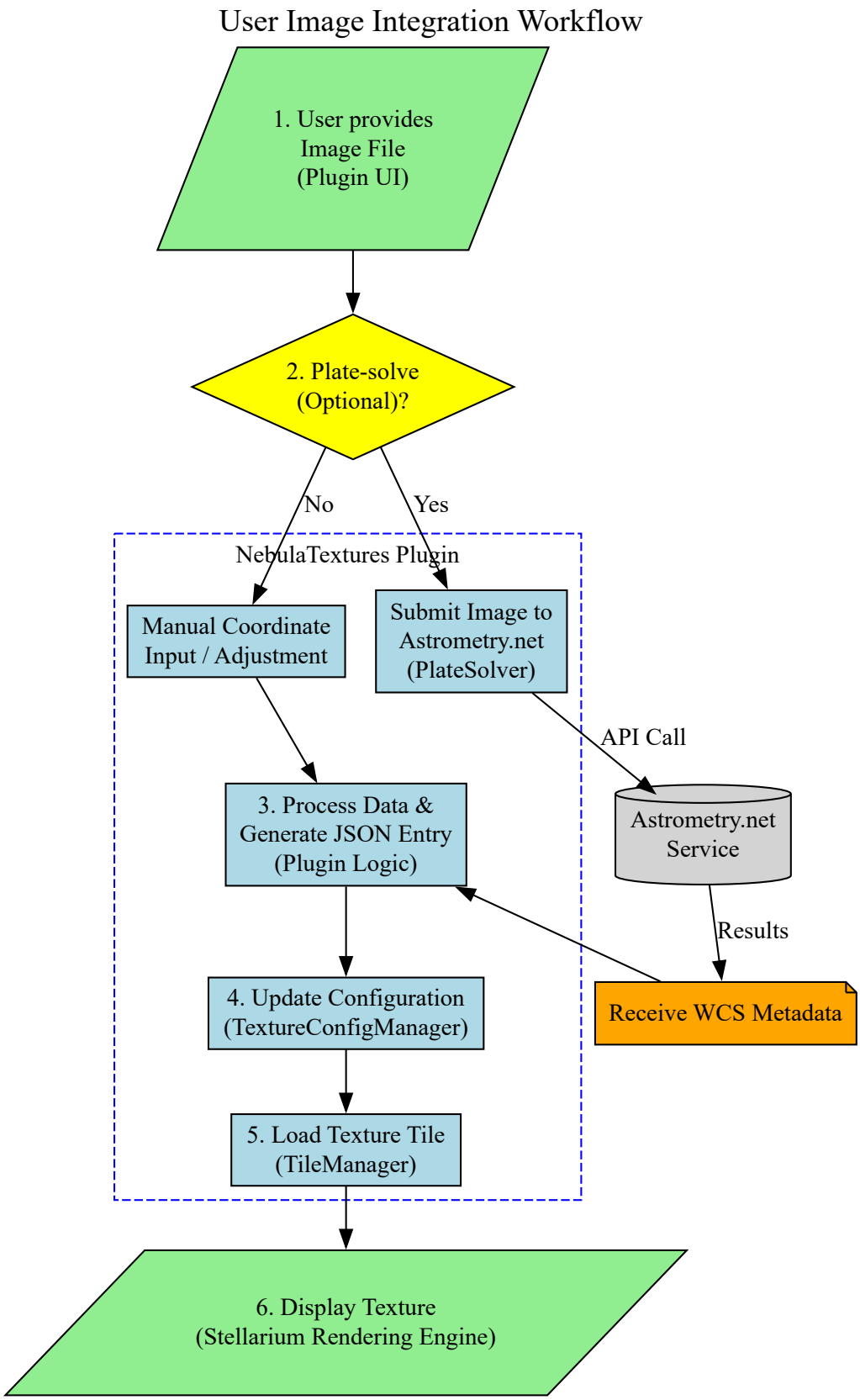

Figure 1: Figure 1: User image integration workflow of NebulaTextures Plugin

Astrometric Algorithms

Astrometric Projection: Pixel-to-Celestial Mapping

NebulaTextures implements standard TAN (gnomonic) projection to convert pixel coordinates (X,Y) into celestial coordinates (α,δ):

1. Intermediate Plane Coordinates:

$$\xi = CD_{1,1}(X - CRPIX_1) + CD_{1,2}(Y - CRPIX_2)$$

$$\eta = CD_{2,1}(X - CRPIX_1) + CD_{2,2}(Y - CRPIX_2)$$

where $CRPIX_1, CRPIX_2$ are the reference pixel coordinates and $CD_{i,j}$ are the coordinate transformation matrix elements.

2. Projection Geometry (for TAN projection):

$$r = \sqrt{\xi^2 + \eta^2}, \quad c = \arctan(r)$$

3. Declination:

$$\sin(\delta) = \cos(c)\sin(\delta_0) + \frac{\eta \sin(c)\cos(\delta_0)}{r}$$

$$\delta = \arcsin(\text{clamp}(\sin(\delta), -1, 1))$$

4. Right Ascension:

$$\Delta\alpha = \text{atan2}(\xi \sin(c), r\cos(\delta_0)\cos(c) - \eta\sin(\delta_0)\sin(c))$$

$$\alpha = (\alpha_0 + \Delta\alpha) \bmod 2\pi$$

where (α_0, δ_0) are the celestial coordinates of the reference pixel (usually the image center) and atan2 is the two-argument arctangent.

5. Normalization: RA is typically expressed in degrees in the [0, 360) interval, and Dec in degrees in the [−90, 90] interval.

Estimating Image Center from Corner Coordinates

When WCS data is unavailable, the image center can be estimated from its four corner coordinates (α_i, δ_i) by averaging their unit vectors:

1. **Compute and average unit vectors:**

$$(\bar{x}, \bar{y}, \bar{z}) = \frac{1}{4}\sum_{i=1}^{4}(\cos\delta_i\cos\alpha_i,\ \cos\delta_i\sin\alpha_i,\ \sin\delta_i)$$

2. **Convert back to spherical coordinates:**

DRAFT

$$RA = \text{atan2}(\bar{y}, \bar{x}) \times \frac{180}{\pi} \bmod 360$$

$$Dec = \arcsin\left(\frac{\bar{z}}{\sqrt{\bar{x}^2 + \bar{y}^2 + \bar{z}^2}}\right) \times \frac{180}{\pi}$$

This method is more robust for large fields or distorted projections compared to simple arithmetic averaging of RA and Dec.

Core Mechanics

NebulaTextures implements three key technical improvements aimed at addressing core challenges in dynamic texture management and visualization.

Rendering Lifecycle Integration

Injecting arbitrary textures at runtime into an engine originally designed for static asset loading poses significant OpenGL context and resource lifecycle challenges. We introduce a **pre-emptive injection** mechanism, implemented via a static macro hook in the plugin's core, that aligns the plugin's initialization and texture registration sequence precisely with Stellarium's module loading stages. This guarantees that custom textures are fully registered before the primary OpenGL context and texture managers finalize, thereby eliminating runtime conflicts, rendering artifacts, and undefined behaviors.

Hierarchical Texture Management

To support arbitrarily large or high-resolution user mosaics without sacrificing performance, we adapt multilevel tiling strategies to deep-sky imaging. The JSON-based `MultiLevelJsonBase` schema lets users describe parent and child tile hierarchies across varying levels of detail. Our `TileManager` then performs **lazy loading**, dynamically streaming only those tiles intersecting the current field of view and zoom level, ensuring both scalability and interactive framerates. Runtime APIs (`setVisible`, `getVisible`) grant fine-grained control over individual tiles or entire texture groups, facilitating side-by-side comparisons and curated presentations.

Overlay Conflict Detection and Blending

When user textures overlap Stellarium's built-in deep-sky object (DSO) layers, naïve rendering can obscure critical detail. We address this with a geometry-based conflict resolution algorithm. Convex-hull intersection checks in celestial coordinates identify overlapping regions, and the plugin dynamically suppresses the corresponding default textures. This **intelligent overlay blending** prioritizes user-provided data, preserves visual clarity, and enhances comparative analysis without manual intervention.

Applications

NebulaTextures simplifies the integration of custom DSO imagery into Stellarium, enabling tailored sky visualizations across multiple domains:

- **Education:** Facilitates display of student astrophotography, concept-specific overlays, and custom guided tours aligned with curricula.
- **Outreach:** Enhances planetarium shows and public programs with local imagery, historical charts, or cultural sky maps for more engaging, audience-specific content.

DRAFT

- **Research:** Supports visual comparison of observations with models, planning of observations, and presentation of preliminary results. Plate-solving via Astrometry.net streamlines image alignment and evaluation.

The plugin encourages community contributions, fostering a shared library of solved and formatted sky textures within Stellarium.

Acknowledgements

We thank the Stellarium team for their extensible plugin framework and their thoughtful guidance provided in the pull request, and the Astrometry.net project for their essential plate-solving service.

References

DRAFT